ColdCard hack: machine-learning detection of theft transactions

Course: Blockchain Analysis | **Assignment:** ColdCard – ML | **Work:** individual | **Continues:** ColdCard – Statistical Analyses

1. Objective

Train machine-learning models that detect the ColdCard theft transactions and evaluate them with the standard tools of chapter 3 of the scikit-learn user guide (model selection and evaluation). The data are the 98 blocks used in the previous exercise, parsed into transaction, input and output tables (see that report for the parsing and its validation).

2. Problem definition

Unit and label. Each row is one transaction. The label is 1 if the transaction pays the collection address given in the previous assignment and does not spend from it, and 0 otherwise. In the previous exercise this gave **504 theft transactions**. Coinbase transactions are excluded because they have no real inputs and cannot be thefts. The dataset has **493,473 transactions**, so positives are **0.102%**: the problem is extremely imbalanced.

Consequence for evaluation. A model that always answers "not theft" reaches 99.9% accuracy while finding nothing, so accuracy is not used. The report uses precision, recall, F1, the confusion matrix, and the ROC and precision-recall curves.

Avoiding leakage. The label is defined by the destination address, so no feature may reveal it. Features describe only the *shape* of the transaction. The destination addresses, the block height and the timestamp were never used as features.

3. Features

Eighteen features were built for each transaction, in three groups.

Group	Features
Transaction	`n_in`, `n_out`, `size`, `weight`, `fee`, `fee_rate` (satoshis per virtual byte, fee / (weight/4)), `ver`, `lock_time`
Inputs (aggregated per transaction)	`in_sum`, `in_mean`, `in_min`, `in_max`, `in_std`, `in_n_addr` (distinct input addresses), `in_share_bc1q` (share of inputs from `bc1q` addresses), `seq_min`, `seq_max` (input sequence numbers)
Outputs	`out_share_max` (largest output divided by the sum of outputs)

Missing values. `out_share_max` is undefined (0/0) for 222 transactions whose single output has value 0. None of them is a theft, and they were filled with 0 (a constant, so nothing is learned from the data).

`lock_time` removed. In the first run all 18 features were used. `lock_time` is often set by wallets to the current block height, so it acts as a timestamp, which breaks the rule above. It was removed from the later experiments (cross-validation, tuning and final models), which use 17 features.

4. Experimental design

- **Split.** Stratified random split, 70% training and 30% test (`random_state=42`): 345,431 training transactions (353 thefts) and 148,042 test transactions (151 thefts). Stratification keeps the 0.1% share in both parts. A temporal split was not used because the thefts caught in the test set lie in four consecutive blocks (see section 7), so a temporal split would leave almost no thefts in a later test set.
- **Imbalance.** All models use class weights (`"balanced"`, or `"balanced_subsample"` for the forest).
- **Models.** A dummy baseline (always predicts "not theft"), logistic regression (with standardized features), a decision tree and a random forest.
- **Evaluation tools (chapter 3).** Confusion matrix, `classification_report` (precision, recall, F1), ROC AUC, average precision (PR AUC), stratified 5-fold cross-validation, `GridSearchCV`, and a decision threshold selected on out-of-fold predictions.

Metrics. Precision = TP/(TP+FP): how many alerts are real. Recall = TP/(TP+FN): how many thefts are caught. F1 is their harmonic mean. ROC AUC and PR AUC judge the ranking of scores without a threshold; with 0.1% positives, the PR AUC is the more informative (a random ranking scores about 0.001 on it).

5. Results

5.1 Baseline and first run (18 features, default settings, test set)

The dummy model reaches an accuracy of 0.99898 and a recall of 0: its confusion matrix is `[[147891, 0], [151, 0]]`, so it misses all 151 thefts.

Model	TP	FP	FN	Precision	Recall	F1	ROC AUC	PR AUC
Logistic regression	150	951	1	0.136	0.993	0.240	0.997	0.147
Decision tree (depth 6)	150	137	1	0.523	0.993	0.685	0.996	0.523
Random forest (200 trees)	88	82	63	0.518	0.583	0.548	0.990	0.540

Logistic regression has the highest ROC AUC and yet 951 false alarms for 150 hits, which shows why ROC AUC is misleading here. The forest, with the default threshold of 0.5, misses 63 thefts.

5.2 Cross-validation (17 features, stratified 5-fold, mean ± standard deviation)

Model	Precision	Recall	F1	PR AUC
Logistic regression	0.140 ± 0.008	0.992 ± 0.007	0.245 ± 0.011	0.154 ± 0.005
Decision tree (depth 6)	0.528 ± 0.048	0.982 ± 0.008	0.686 ± 0.042	0.529 ± 0.027
Random forest	0.537 ± 0.023	0.615 ± 0.052	0.573 ± 0.032	0.522 ± 0.022

The results agree with the single split, so it was not luck. The tree and the forest have the same PR AUC within the spread: they rank transactions equally well, and the forest's low recall comes from its default cut-off.

5.3 Hyperparameter search and decision threshold

`GridSearchCV` was run on the training data only, scored by average precision. For the tree: `max_depth` in {3, 4, 6, 8, 12} and `min_samples_leaf` in {1, 5, 20, 100} with 5-fold cross-validation; best: depth 12, leaf 20 (cross-validated PR AUC 0.587). For the forest (100 trees): `max_depth` in {8, 12} and `min_samples_leaf` in {5, 20} with 3-fold cross-validation; best: depth 8, leaf 20 (PR AUC 0.600).

The decision threshold was then chosen to maximize F1 on out-of-fold probabilities of the training set (5-fold), and the model was evaluated once on the test set: threshold 0.994 for the tree and 0.648 for the forest.

Final model (test set)	TP	FP	FN	Precision	Recall	F1	PR AUC
Decision tree, tuned + threshold	150	139	1	0.519	0.993	0.682	0.576
Random forest, tuned + threshold	150	131	1	0.534	0.993	0.694	0.534

Confusion matrices: tree `[[147752, 139], [1, 150]]`, forest `[[147760, 131], [1, 150]]`. Tuning and the threshold raised the forest's recall from 0.583 to 0.993 and its F1 from 0.548 to 0.694. The tree's threshold of 0.994 is extreme because tree probabilities are almost always 0 or 1, so the forest is the cleaner final model. Its cross-validated PR AUC (0.600) is higher than the test value (0.534); with only 151 thefts in the test set, such a gap is within the noise.

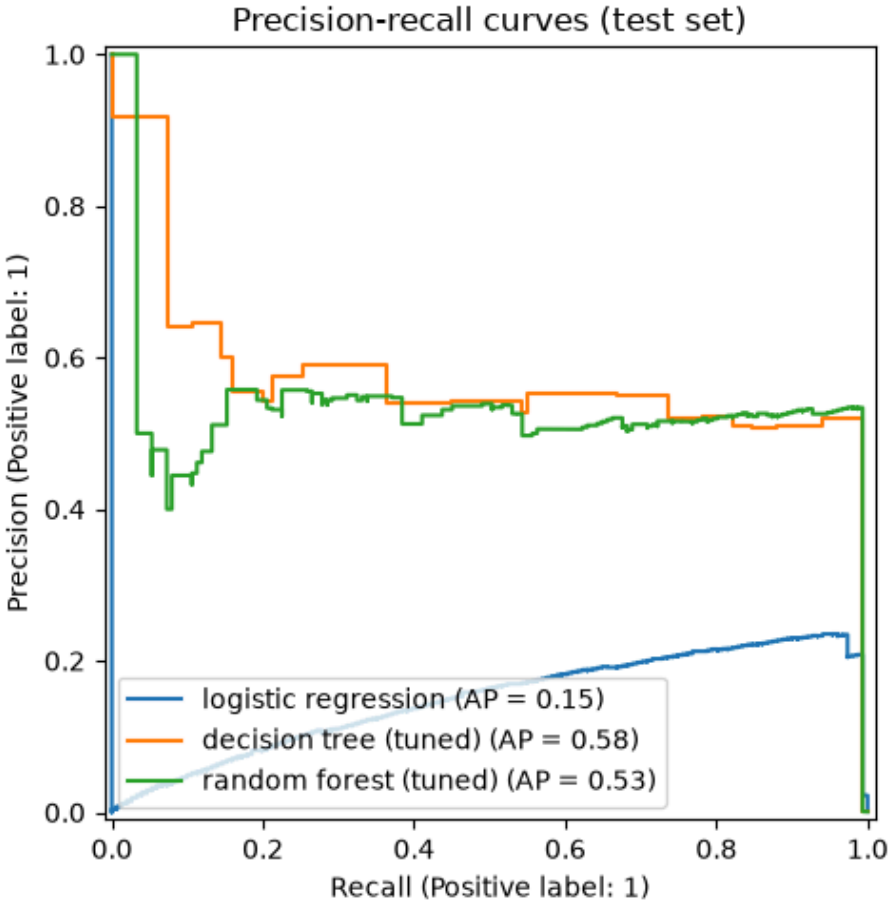


Figure 1. Precision-recall curves on the test set (average precision: logistic regression 0.15, tuned tree 0.58, tuned forest 0.53).

The tree and the forest curves stay on a plateau of precision around 0.5 to 0.55 up to a recall close to 1, while logistic regression stays near 0.2.

6. What the models use

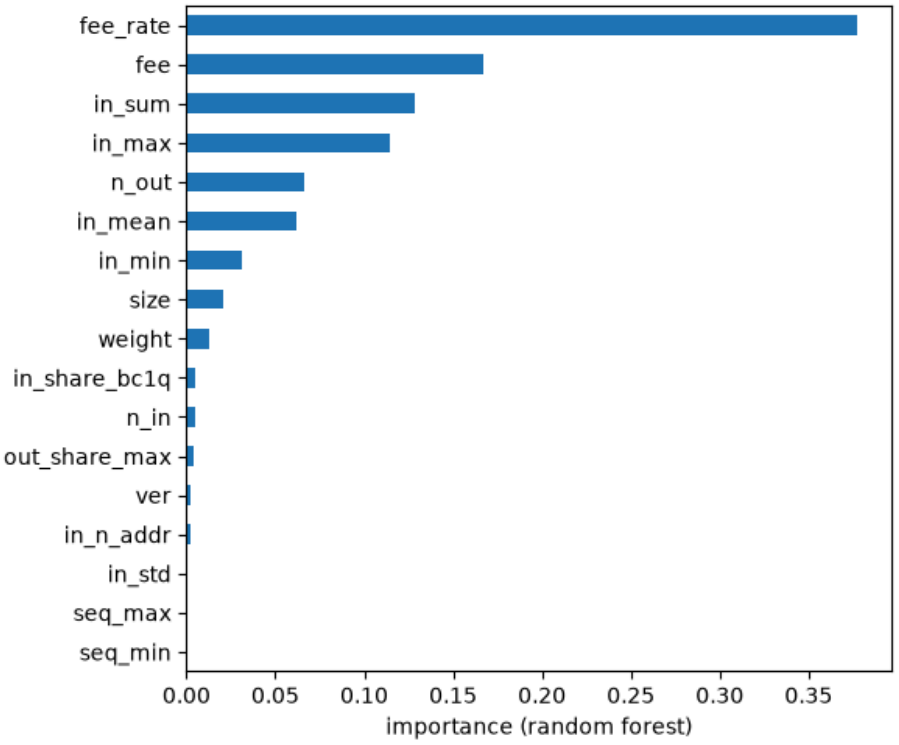


Figure 2. Feature importance of the final random forest.

The largest importances are `fee_rate` (0.378), `fee` (0.167), `in_sum` (0.128), `in_max` (0.114), `n_out` (0.067), `in_mean` (0.062), `in_min` (0.032) and `size` (0.021). The two fee features together carry 0.545, and the four input-value features 0.336. The root of the depth-6 tree is also `fee_rate` (importance 0.98), with the theft branch at `30.04 < fee_rate <= 30.33` and a large input value (above about 15,000,000 satoshis, 0.15 BTC). Theft transactions are mostly single-input, single-output sweeps with a median of about 0.41 BTC and a median fee rate of about 30.1 satoshis per virtual byte, against 0.0005 BTC and 0.45 sat/vB for the other transactions.

7. Error analysis: the false alarms

All good models end at about 150 of 151 thefts found and 130 to 140 false alarms, which suggests a limit set by the features, not by the model. The false alarms of the depth-6 tree (137 in the test set) have the same medians as the true thefts: one input, one output, a `bc1q` input, the same sequence number, and the same fee rate (30.137 sat/vB, with the two values 30.206 and 30.137 dominating in both groups). Their block heights are concentrated around block 960185, three blocks before the thefts (the 150 thefts caught by the tree lie in blocks 960188 to 960191), and one address receives 120 of the outputs of these false alarms.

Hypothesis (not tested): many of these false alarms are sweeps by the same tool to a different address, so they would be thefts that our label does not contain, and the real precision would be higher than measured. This was not verified, and the owner of that address was not investigated.

8. Limitations

- **Label.** Only payments to a single collection address are positive. Similar transactions to other addresses are counted as negatives, which lowers the measured precision if they are really thefts.

- **One attack, a short window.** The theft transactions span block heights 960188 to 960793, but the thefts caught in the test set all lie in four blocks (960188 to 960191), and the test set has only 151 of them. All metrics are noisy: the cross-validation standard deviations (up to 0.05) show it.
- **Random split.** Transactions from the same blocks appear in both training and test, so the results measure detection within this attack, not generalization to other attacks or periods. A temporal split was not possible with so few later thefts.
- **A fingerprint of the tool.** The models rely mainly on fee rate and the size of the input. A fixed fee rate looks like a feature of the sweeping software, so these models would probably not transfer to other thefts. *(Hypothesis, not tested.)*
- **Threshold and tuning.** The threshold was chosen on training data and the grids were small. ROC AUC was computed only for the first run.
- **First run versus final models.** The first-run models include `lock_time`; the later ones do not.
- **Block selection.** The 98 provided blocks are not contiguous, and the selection criterion is unknown (see the previous report).

9. Conclusion

With features that describe only the shape of a transaction, a tuned random forest with a chosen threshold finds 150 of the 151 test thefts (recall 0.993) with a precision of 0.534 (F1 0.694). Accuracy and ROC AUC are not informative under 0.1% positives, which logistic regression illustrates: ROC AUC of 0.997, precision of 0.136. The remaining false alarms look like the thefts in every feature used, so further gains would need other kinds of information (for example the history of the addresses) and not a different model.

Appendix A. Code

The code ran as notebook cells in Python (pandas, numpy, scikit-learn, matplotlib). It follows the parsing and cleaning cells of the statistical report, which create `txs`, `ins`, `outs` and `inflow_tx` (the 504 theft transactions).

A.1 Dataset: features and label

```
import pandas as pd, numpy as np

theft = inflow_tx                                    # the 504 transactions paying the collection
address
tx_ml = txs[~txs["is_coinbase"]].copy()              # coinbase: no real inputs, never theft

g = ins.groupby("txid")
ins["is_bc1q"] = ins["addr"].str.startswith("bc1q")
f_in = pd.DataFrame({
    "in_sum":   g["value"].sum(),
    "in_mean":  g["value"].mean(),
    "in_min":   g["value"].min(),
    "in_max":   g["value"].max(),
    "in_std":   g["value"].std().fillna(0),
    "in_n_addr": g["addr"].nunique(),
    "in_share_bc1q": ins.groupby("txid")["is_bc1q"].mean(),
    "seq_min":  g["sequence"].min(),
    "seq_max":  g["sequence"].max(),
})

o = outs.groupby("txid")["value"]
f_out = pd.DataFrame({"out_share_max": o.max() / o.sum()})

df = tx_ml.set_index("txid")[["n_in", "n_out", "size", "weight", "fee", "lock_time", "ver"]]
```

```
df = df.join(f_in).join(f_out)
df["fee_rate"] = df["fee"] / (df["weight"] / 4)      # satoshis per virtual byte
df["y"] = df.index.isin(theft).astype(int)

print(df.shape, df["y"].sum(), round(df["y"].mean(), 5))      # (493473, 19) 504 0.00102
print(df.loc[df["out_share_max"].isna(), "y"].sum())          # 0 thefts among the 222 missing values
df["out_share_max"] = df["out_share_max"].fillna(0)
```

A.2 Split and dummy baseline

```
from sklearn.model_selection import train_test_split
from sklearn.dummy import DummyClassifier
from sklearn.metrics import accuracy_score, confusion_matrix, recall_score

X = df.drop(columns="y")
y = df["y"]
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, stratify=y, random_state=42)
print(X_train.shape, X_test.shape, y_train.sum(), y_test.sum())   # (345431, 18) (148042, 18) 353 151

dummy = DummyClassifier(strategy="most_frequent").fit(X_train, y_train)
pred = dummy.predict(X_test)
print(accuracy_score(y_test, pred), recall_score(y_test, pred))    # 0.99898 0.0
print(confusion_matrix(y_test, pred))
```

A.3 First run: three models

```
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import classification_report, average_precision_score, roc_auc_score

models = {
    "logreg": make_pipeline(StandardScaler(), LogisticRegression(class_weight="balanced",
max_iter=1000)),
    "tree":   DecisionTreeClassifier(max_depth=6, class_weight="balanced", random_state=42),
    "forest": RandomForestClassifier(n_estimators=200, class_weight="balanced_subsample",
                                     n_jobs=-1, random_state=42),
}

results = {}
for name, m in models.items():
    m.fit(X_train, y_train)
    pred = m.predict(X_test)
    proba = m.predict_proba(X_test)[:, 1]
    results[name] = (m, pred, proba)
    print("=====", name)
    print(confusion_matrix(y_test, pred))
    print(classification_report(y_test, pred, digits=3))
    print("ROC AUC", round(roc_auc_score(y_test, proba), 4),
          "PR AUC", round(average_precision_score(y_test, proba), 4))
```

A.4 Inspecting the tree and its false alarms

```
from sklearn.tree import export_text

tree = results["tree"][0]
print(export_text(tree, feature_names=list(X.columns), max_depth=4))
imp = pd.Series(tree.feature_importances_, index=X.columns).sort_values(ascending=False)
print(imp.head(8))

pred_t = results["tree"][1]
fp = X_test[(pred_t == 1) & (y_test == 0)]
tp = X_test[(pred_t == 1) & (y_test == 1)]
cols = ["n_in", "n_out", "in_sum", "fee_rate", "in_share_bc1q", "seq_min", "in_n_addr"]
print(len(fp), len(tp))
print(pd.concat([fp[cols].median().rename("false_alarm"),
                 tp[cols].median().rename("true_theft")], axis=1))
```

```
fp_out = outs[outs["txid"].isin(fp.index)]              # analysis only, not used as a feature
print(len(fp), fp_out["addr"].nunique(), fp_out["addr"].value_counts().head(5).values)
h = txs.set_index("txid")["height"]
print(h.loc[fp.index].describe()); print(h.loc[tp.index].describe())
print(fp["fee_rate"].round(3).value_counts().head(5))
print(tp["fee_rate"].round(3).value_counts().head(5))
```

A.5 Cross-validation without `lock_time`

```
from sklearn.model_selection import StratifiedKFold, cross_validate

feat = [c for c in X.columns if c != "lock_time"]          # lock_time acts as a timestamp
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
scoring = {"precision": "precision", "recall": "recall", "f1": "f1", "pr_auc": "average_precision"}

for name, m in models.items():
    r = cross_validate(m, X[feat], y, cv=cv, scoring=scoring, n_jobs=1)
    print(name, {k: (round(r[f"test_{k}"].mean(), 3), round(r[f"test_{k}"].std(), 3)) for k in scoring})
```

A.6 Hyperparameter search

```
from sklearn.model_selection import GridSearchCV

Xtr, Xte = X_train[feat], X_test[feat]

grid_tree = GridSearchCV(
    DecisionTreeClassifier(class_weight="balanced", random_state=42),
    {"max_depth": [3, 4, 6, 8, 12], "min_samples_leaf": [1, 5, 20, 100]},
    scoring="average_precision", cv=5, n_jobs=-1)
grid_tree.fit(Xtr, y_train)
print(grid_tree.best_params_, round(grid_tree.best_score_, 3))     # depth 12, leaf 20, 0.587

grid_rf = GridSearchCV(
    RandomForestClassifier(n_estimators=100, class_weight="balanced_subsample",
                           n_jobs=-1, random_state=42),
    {"max_depth": [8, 12], "min_samples_leaf": [5, 20]},
    scoring="average_precision", cv=3)
grid_rf.fit(Xtr, y_train)
print(grid_rf.best_params_, round(grid_rf.best_score_, 3))         # depth 8, leaf 20, 0.600
```

A.7 Decision threshold and final evaluation

```
from sklearn.model_selection import cross_val_predict
from sklearn.metrics import precision_recall_curve

final = {}
for name, est in [("tree", grid_tree.best_estimator_), ("forest", grid_rf.best_estimator_)]:
    oof = cross_val_predict(est, Xtr, y_train, cv=5, method="predict_proba")[:, 1]
    p, r, t = precision_recall_curve(y_train, oof)
    f1 = 2 * p * r / (p + r + 1e-12)
    thr = t[np.argmax(f1[:-1])]
    est.fit(Xtr, y_train)
    proba = est.predict_proba(Xte)[:, 1]
    pred = (proba >= thr).astype(int)
    final[name] = (est, thr, proba, pred)
    print("=====", name, "threshold", round(thr, 3))
    print(confusion_matrix(y_test, pred))
    print(classification_report(y_test, pred, digits=3))
    print("PR AUC", round(average_precision_score(y_test, proba), 4))
```

A.8 Figures

```
from sklearn.metrics import PrecisionRecallDisplay
import matplotlib.pyplot as plt, os
os.makedirs("figures", exist_ok=True)

fig, ax = plt.subplots(figsize=(6, 5))
for name, proba in [("logistic regression", results["logreg"][2]),
                    ("decision tree (tuned)", final["tree"][2]),
                    ("random forest (tuned)", final["forest"][2])]:
```

```
    PrecisionRecallDisplay.from_predictions(y_test, proba, name=name, ax=ax)
ax.set_title("Precision-recall curves (test set)")
fig.tight_layout(); fig.savefig("figures/pr_curves.png", dpi=200)

imp = pd.Series(final["forest"][0].feature_importances_, index=feat).sort_values()
fig, ax = plt.subplots(figsize=(6, 5))
imp.plot.barh(ax=ax); ax.set_xlabel("importance (random forest)")
fig.tight_layout(); fig.savefig("figures/importance.png", dpi=200)
print(imp.sort_values(ascending=False).head(8).round(3))
```